
Software Requirements Specification

for

AI Legal Assitant

Version <1.3>

Prepared by

Group Name: 15

Soham Sunil Malvadkar	22141229	soham05malvadkar@gmail.com
Sanket Nivas Chopade	22141234	sanketchopade2025@gmail.com
Abhijeet Vilas Ganjave	22141236	abhijeetganjave04@gmail.com
Akash Ananda Khot	22141244	akkikhot2309@gmail.com
Sanket Nandkumar Adake	22143248	sankuash143@gmail.com

Instructor: Dr. S. A. Thorat

Course: Seminar

Lab Section: <place your lab section here>

Teaching Assistant: <place your TA's name here>

Date: <place the date of submission here>

Contents

CONTENTS	II
REVISIONS	III
1 INTRODUCTION	1
1.1 DOCUMENT PURPOSE	1
1.2 PRODUCT SCOPE	1
1.3 INTENDED AUDIENCE AND DOCUMENT OVERVIEW.....	1
1.4 DEFINITIONS, ACRONYMS AND ABBREVIATIONS.....	2
1.5 DOCUMENT CONVENTIONS	2
1.6 REFERENCES AND ACKNOWLEDGMENTS.....	3
2 OVERALL DESCRIPTION	4
2.1 PRODUCT OVERVIEW	4
2.2 PRODUCT FUNCTIONALITY	4
2.3 DESIGN AND IMPLEMENTATION CONSTRAINTS	5
2.4 ASSUMPTIONS AND DEPENDENCIES	6
3 SPECIFIC REQUIREMENTS	7
3.1 EXTERNAL INTERFACE REQUIREMENTS	7
3.2 FUNCTIONAL REQUIREMENTS	8
3.3 USE CASE MODEL.....	10
4 OTHER NON-FUNCTIONAL REQUIREMENTS	12
4.1 PERFORMANCE REQUIREMENTS.....	12
4.2 SAFETY AND SECURITY REQUIREMENTS.....	12
4.3 SOFTWARE QUALITY ATTRIBUTES	13
5 OTHER REQUIREMENTS	15
APPENDIX A – DATA DICTIONARY	17
APPENDIX B - GROUP LOG	19

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0 (Draft 1)	Sanket N. Chopade	Initial draft of SRS covering Introduction and Overall Description sections.	12/10/2025
1.1 (Draft 2)	Soham S. Malvadkar, Abhijeet V. Ganjave	Added Functional Requirements, Use Case Model, and updated diagrams.	16/10/2025
1.2 (Draft 3)	Akash A. Khot, Sanket N. Adake	Incorporated Non-Functional Requirements and completed Appendices.	22/10/2025
1.3 (Final Version)	Entire Team	Final proofreading, formatting, and review for submission.	26/10/2025

1 Introduction

The AI Legal Assistant project is an intelligent legal support system designed to simplify access to legal knowledge and automate document-based legal query resolution. This system integrates three major components — **Document Analysis**, **Legal Chatbot**, and **Lawyer Finder**—to assist users in understanding legal information, analyzing uploaded documents, and connecting with professional lawyers.

This section introduces the purpose, scope, and audience of this Software Requirements Specification (SRS) document, along with the key conventions, acronyms, and references used throughout.

1.1 Document Purpose

This SRS document specifies the functional and non-functional requirements for the **AI Legal Assistant** software, Version 1.0. It describes the behavior, constraints, and design goals of the system and serves as a communication bridge between the development team and the stakeholders.

The primary purpose of this document is to ensure that all project participants—developers, testers, faculty supervisors, and end-users—share a mutual understanding of the system's features and expectations. This SRS covers all three modules: **Document Analysis**, **Legal Chatbot**, and **Lawyer Finder**. It defines how each component will operate and interact to deliver an intelligent, user-friendly legal assistance platform.

1.2 Product Scope

The AI Legal Assistant is an AI-driven web application aimed at democratizing access to legal knowledge. The system enables users to:

- **Upload and analyze legal documents** using advanced text extraction (Ollama DeepSeek R1 1.5B para model) and semantic search (FAISS).
- **Interact with a legal chatbot** trained on curated datasets related to marriage and employee law using Hugging Face and Groq APIs.
- **Find lawyers** based on expertise, location, and ratings through an integrated database.

The product benefits users by significantly reducing time spent on legal research, simplifying legal jargon, and offering reliable, context-aware responses. It is designed to assist both general users and law professionals by providing quick, accurate, and affordable access to legal information.

1.3 Intended Audience and Document Overview

This document is primarily intended for:

- **Professors and evaluators** – to assess the project's requirements and technical design.
- **Developers and testers** – to implement and verify system components according to specified requirements.
- **End-users (clients)** – to understand the product's functionality and expected performance.

The SRS is organized into multiple sections for clarity:

- **Section 1:** Introduction to the system and document conventions.
- **Section 2:** Overall description of the product's functionality and system environment.
- **Section 3:** Specific requirements detailing system behavior, use cases, and interfaces.
- **Section 4:** Non-functional requirements, such as performance and security.
- **Section 5:** Other supporting requirements and appendices.

Readers are advised to begin with Sections 1 and 2 for a conceptual overview before reviewing technical and functional details in Sections 3 and 4.

1.4 Definitions, Acronyms and Abbreviations

Term / Acronym	Description
AI	Artificial Intelligence
API	Application Programming Interface
FAISS	Facebook AI Similarity Search
GUI	Graphical User Interface
LLM	Large Language Model
OCR	Optical Character Recognition
SRS	Software Requirements Specification
UI	User Interface
KB	Knowledge Base
NLP	Natural Language Processing
Groq API	API used for high-speed model inference
Ollama	Framework for running large language models locally
DeepSeek R1 1.5B	Model used for document text extraction
Hugging Face	Open-source platform for training and deploying AI models

1.5 Document Conventions

This SRS document follows the IEEE Standard 830-1998 for Software Requirements Specifications.

- **Font Style:** Arial
- **Font Size:** 11-point for body text, 12-point bold for section headings
- **Line Spacing:** Single-spaced with 1-inch margins
- **Formatting Conventions:**
 - *Italics* are used for placeholder or comment text.
 - **Bold** text emphasizes important terms or section titles.
 - Tables and lists are used for clarity in describing requirements and definitions.
- **Naming Conventions:** Module names follow PascalCase (e.g., DocumentAnalysis, LegalChatbot).

1.6 References and Acknowledgments

1. IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications.
2. Groq API Documentation — <https://groq.com/>
3. Hugging Face Documentation — <https://huggingface.co/docs>
4. FAISS Library Documentation — <https://faiss.ai/>
5. Ollama Model Documentation — <https://ollama.ai>
6. Project guidance and supervision by **Information Technology Department, Government College of Engineering, Karad.**

2 Overall Description

2.1 Product Overview

The AI Legal Assistant is a new, self-contained software system that combines advanced Natural Language Processing (NLP), semantic search, and AI-driven reasoning to assist users with legal document analysis and question answering.

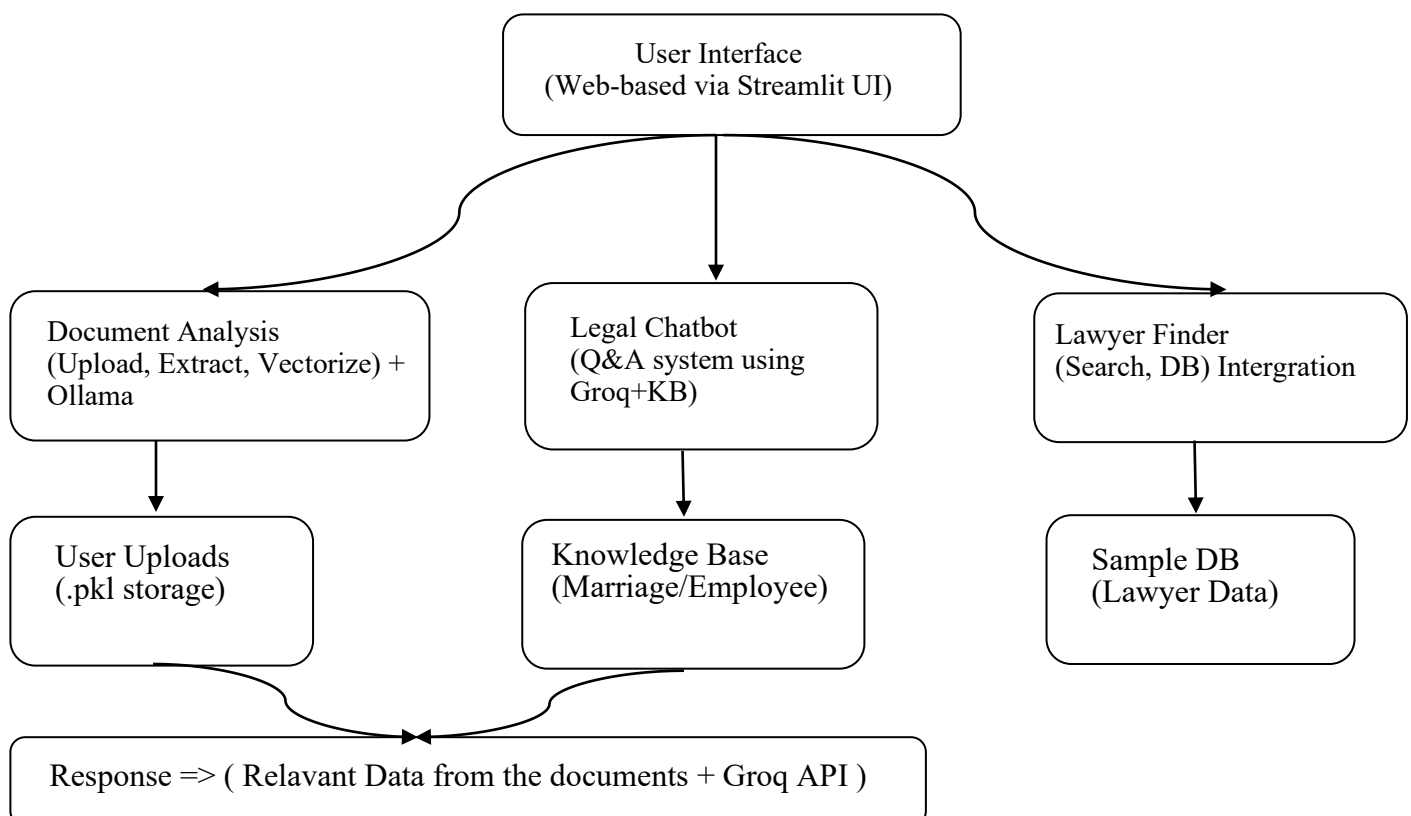
It is not a follow-on product but an innovative legal automation tool designed to bridge the gap between ordinary users and complex legal systems. It simplifies legal research, automates text extraction from uploaded documents, and generates contextually accurate responses using modern AI models.

The system consists of three primary modules:

1. **Document Analysis** – Allows users to upload legal documents (PDF, DOCX, TXT) which are processed using **Ollama Deepseek R1 1.5B** for text extraction. The extracted text is vectorized using FAISS and stored for semantic retrieval.
2. **Legal Chatbot** – Provides conversational interaction in the domains of Marriage and Employment Laws. Pre-trained models (from Hugging Face) and the **Groq LLaMA 3.1-8B-Instant** model generate natural, reliable responses to user queries.
3. **Lawyer Finder** – Enables users to search and filter lawyers based on expertise, location, and experience using a sample database, enhancing access to professional consultation.

High-Level System Context Diagram

Below is a conceptual illustration of the overall system architecture:



This architecture ensures modularity—each component operates independently but communicates through shared APIs and vector databases, enabling efficient retrieval and response generation

2.2 Product Functionality

The AI Legal Assistant software performs the following key functions:

- **Document Upload & Processing:**
Users can upload legal documents (PDF/DOCX/TXT). The system extracts text using Ollama Deepseek R1.5B and converts it into embeddings for future reference.
- **Semantic Search & Retrieval (FAISS):**
The system uses FAISS for high-speed, context-aware query retrieval from document embeddings.
- **AI-Powered Legal Chatbot:**
The chatbot handles domain-specific queries related to marriage and employee law. It retrieves relevant knowledge base data and generates contextually correct answers using Groq LLaMA 3.1-8B.
- **Response Generation & Summarization:**
The Groq API ensures grammatically correct, human-like, and legally grounded responses.
- **Lawyer Finder System:**
Displays a list of lawyers along with their expertise, experience, ratings, and contact details. It supports search and filtering options for better user experience.
- **User Interface & Interaction:**
Provides a simple Streamlit-based web UI for uploading documents, chatting, and finding lawyers.
- **Knowledge Base Maintenance:**
Stores domain-specific trained data in organized .pkl or vector format for future retrieval.

2.3 Design and Implementation Constraints

The design and implementation of the AI Legal Assistant are subject to the following constraints:

- **Hardware Limitations:**
 - *Minimum 4 GB RAM and 2 GB disk space.*
 - *Stable internet connection for API calls to Groq and Hugging Face models.*
- **Software Constraints:**
 - *Developed using **Python 3.8+** with **Streamlit framework** for front-end visualization.*
 - *Uses **FAISS** for semantic search, **Ollama** for model inference, and **Groq API** for response generation.*

- Data storage limited to local environment for .pkl files and small SQLite or CSV database for lawyer information.
- **Design Standards:**
 - The project follows the **COMET (Collaborative Object Modeling and Enterprise Transformation)** method for system design.
 - **UML (Unified Modeling Language)** is used for representing use case diagrams, class diagrams, and sequence flows.
 - Reference:
 - COMET Method: “Real-Time Object-Oriented Modeling” by Hassan Gomaa
 - UML Standard: “OMG Unified Modeling Language (UML) Version 2.5”
- **Security Constraints:**
 - Uploaded legal documents are stored locally (not sent to third-party servers).
 - Access control ensures user privacy and prevents data leakage.
- **Programming Standards:**
 - Follows PEP8 for Python coding standards.
 - All API keys and credentials stored securely in a configuration file (config.json).

2.4 Assumptions and Dependencies

The successful operation of this system depends on the following assumptions and dependencies:

Assumptions:

- Users have a stable internet connection for Groq API and Hugging Face model access.
- The uploaded documents are in readable English or regional legal language supported by the OCR/text extraction model.
- The Groq API and Ollama environment remain stable and available during runtime.
- Users provide queries within the legal domain (marriage or employee-related).

Dependencies:

- External dependencies include:
 - **Groq API** for model inference and response generation.
 - **Hugging Face** platform for pretrained models.
 - **FAISS** for vector-based semantic search.
 - **Ollama Deepseek R1 1.5B** for document extraction and chunking.
- The project also depends on Python libraries such as streamlit, faiss, sentence-transformers, and pandas.
- Any downtime in third-party APIs may affect the system's performance temporarily.

3 Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The AI Legal Assistant provides a **web-based graphical user interface (GUI)** developed using the **Streamlit framework**. The interface is designed to be intuitive, accessible, and responsive for desktop and laptop users.

- **Overview of User Interaction:**

Users can:

- **Upload** legal documents in .pdf, .docx, or .txt formats.
- **Ask Questions** through a chatbot-style input box (text-based interaction).
- **View Extracted Document Content** and corresponding AI-generated summaries.
- **Access Lawyer Finder** section to search lawyers by location, expertise, and rating.

Interface Representation (Illustration):

AI Legal Assistant Web Interface
[Upload Section] -> Choose File (PDF/DOCX/TXT) -> [Upload & An-> [Upload & Analyze Button] analyze Button]
[Document Preview Window] -> Shows extracted text or summary
[Ask Query Section] -> "Type your question here..." [Send] -> Chatbot Response Display
[Lawyer Finder Section] -> Search by Expertise / Location / Rating -> Lawver Profiles displayed in tabular/card format

Interaction Mode: Mouse clicks, keyboard text input, and scroll navigation.

Accessibility: Supports simple English labels, tooltips, and visually separated sections.

3.1.2 Hardware Interfaces

The system is designed to run on standard computing devices and does not require any specialized hardware.

Hardware Component	Description
User Device (PC/Laptop)	Runs the Streamlit web interface through a modern browser (Chrome, Edge, Firefox).
Server / Local Host Machine	Hosts the Python backend, FAISS database, and Groq API connection.
Storage Device	Used for saving .pkl vector files, extracted text, and lawyer database.
Internet Connection	Required for API communication with Groq and Hugging Face servers.

All data processing happens on the system's CPU (no dependency on GPU hardware).

3.1.3 Software Interfaces

The AI Legal Assistant interacts with the following software components:

Software Component	Description	Interaction Type
Streamlit	Front-end interface for web-based interaction	Direct (user input/output)
Groq API	Generates contextually correct responses from LLaMA-3.1 model	REST API calls
Ollama	Extracts and processes document text using DeepSeek R1.5B model	Local runtime
FAISS	Manages vector storage and semantic search	Python library integration
Hugging Face Models	Pretrained models for marriage and employee law datasets	Model loading
CSV Database	Stores lawyer data and ratings	Local database access
Python Libraries	NumPy, Pandas, Sentence-Transformers, etc.	Internal dependencies

3.2 Functional Requirements

Below is a detailed list of the system's functional requirements:

3.2.1 F1: Document Upload and Extraction

- The system shall allow users to upload documents in .pdf, .docx, or .txt formats.
- The system shall use **Ollama DeepSeek R1.5B** to extract and preprocess the document text.
- The system shall save processed embeddings as .pkl files inside the *userupload/* directory.

3.2.2 F2: Document Query Processing

- The system shall allow users to enter queries about the uploaded document.
- The system shall retrieve the most relevant chunks using the **FAISS** similarity search technique.
- The system shall generate a summarized and legally accurate answer via the **Groq API**

3.2.3 F3: Legal Chatbot

- The chatbot shall be capable of answering domain-specific questions related to **Marriage** and **Employee** law.
- The chatbot shall access pre-trained data stored in the *knowledgebase/* folder.
- The chatbot shall use the **Groq LLaMA 3.1-8B-Instant** model for natural response generation.

3.2.4 F4: Lawyer Finder System

- The system shall allow users to search for lawyers by **expertise**, **location**, and **rating**.
- The system shall display results from a sample lawyer database.
- The user shall be able to view lawyer contact information, website, and specialization.

3.2.5 F5: User Interface and Navigation

- The system shall provide a simple, three-section web interface (Document Analysis, Chatbot, Lawyer Finder).
- The interface shall support file upload, chat interaction, and tab-based navigation.

3.2.6 F6: Data Storage and Security

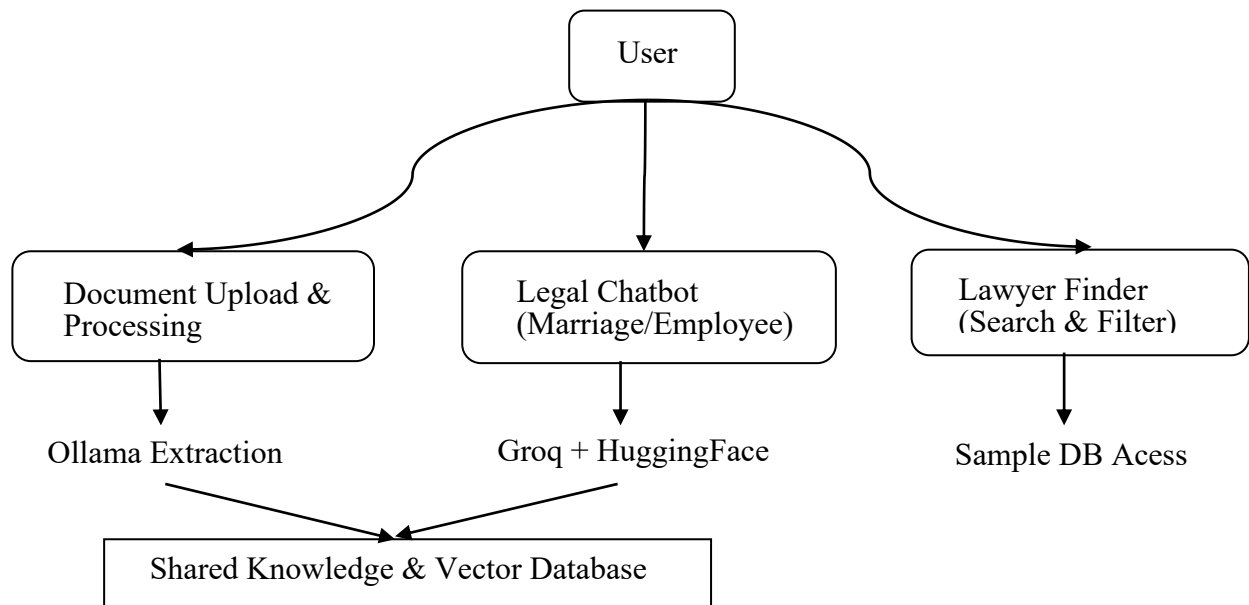
- All uploaded documents shall be stored temporarily in local directories (not shared externally).
- The system shall maintain user privacy and ensure secure handling of sensitive legal data.
- API keys shall be stored securely within `config.json` file.

3.2.7 F7: Error Handling

- The system shall display clear messages for unsupported file formats or failed uploads.
- If an API or model is unavailable, fallback messages shall be shown to the user.

3.3 Use Case Model

3.3.1 Overall Use Case Diagram



Use Case #1: Document Analysis (U1)

Field	Description
Author	Sanket N. Chopade
Purpose	To allow users to upload a document and analyze its content using AI-based extraction.
Requirements Traceability	F1, F2, F6, F7
Priority	High
Preconditions	User must be logged into the application and have a valid document ready for upload.
Postconditions	Extracted text and vector embeddings stored successfully in user_upload/ folder.
Actors	User, Ollama Extraction Model, FAISS Engine
Flow of Events – Basic	1. User uploads document → 2. System extracts text

	→ 3. Embeddings are generated → 4. Stored in local DB.
Alternative Flow	If extraction fails, display “File not supported or extraction error.”
Exceptions	API failure or corrupted file format.
Includes	U2 (Query Answering)
Notes	Extraction must handle both DOCX and PDF with accuracy above 90%.

Use Case #2: Legal Chatbot (U2)

Field	Description
Author	Soham S. Malvadkar
Purpose	To answer user questions related to marriage and employee law using pre-trained knowledge.
Requirements Traceability	F3, F5, F6
Priority	High
Preconditions	Knowledge base data must be preloaded.
Postconditions	Relevant response generated and displayed in chat window.
Actors	User, FAISS Retrieval System, Groq API, Hugging Face Model.
Flow of Events – Basic	1. User types query → 2. System retrieves relevant data → 3. Groq API generates response → 4. Display to user..
Alternative Flow	If no relevant match, show “No relevant law found.”
Exceptions	Groq API timeout or invalid user input.
Includes	—
Notes	Should provide response within 2–3 seconds for user satisfaction.

Use Case #3: Lawyer Finder (U3)

Field	Description
Author	Abhijeet V. Ganjave
Purpose	To help users locate qualified lawyers based on filters like expertise, location, and ratings..
Requirements Traceability	F4, F5
Priority	Medium
Preconditions	Sample lawyer database must be loaded.
Postconditions	Display matching lawyer profiles with contact info.
Actors	User, Database System
Flow of Events – Basic	1. User enters search criteria → 2. System fetches matching lawyers → 3. Results displayed in card/table format.
Alternative Flow	If no match found, show “No lawyer available for this criteria.”
Exceptions	Database not loaded or corrupted.
Notes	User can rate or bookmark lawyers for future reference.

4 Other Non-functional Requirements

4.1 Performance Requirements

The AI Legal Assistant system is expected to deliver fast, consistent, and accurate responses even when handling large legal documents or multiple concurrent queries. Performance directly impacts user trust and usability.

ID	Performance Requirement	Description / Rationale
P1	The system shall process and extract text from a document of size up to 10 MB within 10 seconds .	Ensures acceptable document upload and extraction speed for users.
P2	The FAISS-based retrieval process shall return the top-5 relevant chunks in less than 2 seconds per query.	Maintains real-time performance for question answering.
P3	The Groq API response generation shall not exceed 3 seconds for standard queries (< 100 words).	Ensures conversational fluidity.
P4	The chatbot shall support at least 20 concurrent users without noticeable performance degradation.	Provides scalability for demonstration or small deployment.
P5	The lawyer search query shall execute in under 1 second for databases with ≤ 500 entries.	Provides responsive experience in Lawyer Finder module.
P6	System memory usage shall remain below 1.5 GB during normal operation.	Guarantees efficient local resource utilization.

Rationale:

These performance thresholds ensure that the product remains interactive, practical, and efficient on moderate hardware while demonstrating the capabilities of AI-driven legal systems in real time.

4.2 Safety and Security Requirements

The AI Legal Assistant deals with potentially sensitive legal documents and user queries, thus strong privacy and data-protection mechanisms are essential.

ID	Requirement	Description
S1	Document Privacy	All uploaded legal documents shall be stored locally within the user's session and deleted automatically upon session termination.
S2	The Data Encryption	API keys and configuration files

		(e.g., config.json) shall be encrypted or stored using environment variables to prevent unauthorized access.
S3	Secure API Communication	All external communications with Groq API, Hugging Face, or other services shall use HTTPS with TLS 1.2 or above.
S4	Access Control	Only authorized users (project team or system admin) shall have permission to modify or retrain models.
S5	Error Handling and Fail-Safe	In case of API unavailability or data corruption, the system shall revert to a fallback message without crashing.
S6	Legal Compliance	The system shall comply with general data-protection practices such as anonymization and temporary session storage.
S7	Safety Assurance	The system shall not generate or recommend any legal action; it shall only provide informational responses, displaying a disclaimer for advisory safety.

Rationale:

These safeguards ensure user trust and align the product with ethical and legal data-handling standards. The focus is on confidentiality, integrity, and availability of user information.

4.3 Software Quality Attributes

The following software quality attributes define the expected behavior and standards of the AI Legal Assistant.

4.3.1 Reliability

- The system shall maintain an uptime of $\geq 95\%$ during active sessions.
- Automated exception handling and retry mechanisms will be used for failed API calls.
- Regular testing of document upload and chatbot responses ensures consistent functionality.

4.3.2 Usability

- The user interface shall be minimalistic and intuitive, providing clear instructions and buttons for document upload, chat interaction, and lawyer search.

- Average new user shall understand navigation within **2 minutes** of first use.
- Error messages shall be descriptive (e.g., “Upload only PDF/DOCX/TXT files”) instead of technical jargon.

4.3.3 Maintainability

- The project code shall follow **PEP-8** standards and modular structure (separate files for UI, backend, and models)..
- Each major component (Document Analysis, Chatbot, Lawyer Finder) will be implemented as an independent module for easier debugging and upgrades.
- *Inline comments and version control (GitHub) will support collaborative maintenance.*

4.3.4 Scalability and Extensibility

- The system architecture shall allow new legal domains (e.g., property law, cyber law) to be added by uploading new pre-trained datasets to the knowledge base folder.
- Database schemas and FAISS vector indices are designed to support additional records without modification to the core system.
- *Future deployment on cloud (e.g., Render, AWS) is possible with minor configuration changes.*

4.3.5 Portability

- The application shall run on Windows, Linux, and macOS environments with Python 3.8 +.
- No platform-specific dependencies shall prevent execution on alternate OS.
- Portable virtual environment (`.venv`) and `requirements.txt` ensure reproducibility.

4.3.6 Portability

- The system shall gracefully handle invalid inputs, such as unsupported file types or malformed queries.
- All exceptions will be logged to a `log.txt` file for debugging and auditing.
- Fallback responses will ensure the user experience remains uninterrupted.

5 Other Requirements

5.1 Database Requirements

- The system shall maintain two primary storage components:
 1. **User Upload Storage** – for temporarily storing uploaded document data and extracted .pkl embeddings.
 2. **Lawyer Information Database** – implemented using SQLite or CSV format, containing fields such as:
 - Lawyer ID (Primary Key)
 - Name
 - Expertise (Marriage / Employment / Other)
 - Experience (in years)
 - Location
 - Rating (1–5 scale)
 - Contact Details / Website URL
- The database must support **read, filter, and search** operations in under one second.
- Regular backups or CSV exports should be enabled for maintaining data integrity.

5.2 Internationalization and Localization Requirements

- The system shall initially operate in English, but will be designed for future multi-language support (Marathi, Hindi, Tamil, etc.).
- The translation module will leverage Groq API or compatible translation APIs for query and response translation.
- The chatbot responses will maintain legal accuracy while adapting to regional language syntax.
- Date and numeric formats will follow user locale standards during future expansion.

5.3 Legal and Ethical Requirements

- The system shall include a **disclaimer** stating that all information provided is for **educational and informational purposes only** and not a substitute for professional legal advice.
- Uploaded documents remain confidential and will **not be transmitted** to third-party servers without user consent.
- The system complies with basic **data protection principles**, including local data processing and anonymization where applicable.
- Any use of pre-trained datasets (e.g., Hugging Face models, legal documents) shall respect **open-source licensing** and **data citation norms**.

5.4 Reuse and Scalability Objectives

- The modular design enables reuse of individual components such as:
 - Document Analysis Engine for other domains (e.g., medical, academic).
 - Chatbot Core Framework for integrating different legal areas.
 - FAISS Retrieval System as a standalone search module.
- The architecture supports future integration with external APIs (e.g., Indian Law Information Portals, legal databases).

- The system can scale horizontally by deploying multiple instances for handling increased user requests.

5.5 Future Enhancements

- **Voice Query Input:** Add a speech-to-text module for hands-free operation.
- **Legal Case Summarization:** Expand document analysis to generate concise case summaries with citations.
- **Cloud Integration:** Host embeddings and knowledge base on cloud storage for collaborative access.
- **User Accounts:** Implement authentication and personalized dashboards for repeated users.

5.6 Documentation and Training

- Developer documentation (README + inline code comments) will describe setup, APIs, and dependencies.
- A short **User Manual (PDF)** will be prepared, describing how to upload documents, chat, and search lawyers.
- Team members will provide **demo training** to explain the workflow during project evaluation.

Appendix A – Data Dictionary

The following table defines the key data elements, their types, descriptions, possible values or states, and related operations within the AI Legal Assistant system.

Data Element / Variable	Type / Format	Description / Purpose	Possible Values / States	Related Function / Operation
uploaded_file	File (.pdf/.docx/.txt)	Legal document uploaded by user for analysis	Exists / Not Uploaded / Invalid Format	F1 – Document Upload and Extraction
extracted_text	String	Text extracted from the uploaded document using Ollama model	Null / Valid Text / Error	Text extraction and pre-processing
embedding_vector	List[float]	Numerical vector representation of text generated by FAISS	Stored / Retrieved / Updated	Vector generation and retrieval
user_query	String	Query entered by the user related to legal document or law topic	Text string (up to 200 chars)	F2 – Query Processing, F3 – Chatbot
retrieved_chunks	List[String]	Text segments retrieved as most relevant to query	1–5 Chunks / Empty	FAISS similarity search
ai_response	String	Final response generated by Groq model	Valid Answer / Error / Timeout	F2, F3 – Response Generation
knowledge_base	.pkl / .csv	Stored pre-trained data for legal chatbot	Loaded / Missing / Corrupt	Knowledge base loading
document_status	Enum	Current processing state of uploaded document	Uploaded / Extracted / Analyzed / Failed	State management in document pipeline
lawyer_db	SQLite / CSV	Contains lawyer information including expertise, rating, location	Active / Empty / Error	F4 – Lawyer Finder
lawyer_name	String	Full name of the lawyer	Text	Lawyer Finder – Display info
lawyer_expertise	String	Domain expertise (Marriage / Employee / Others)	Enum values	Filter and search
lawyer_location	String	Geographical location of the	City name / State	Lawyer Finder – Filter

		<i>lawyer</i>		
lawyer_rating	<i>Float (1.0–5.0)</i>	<i>Rating score given to lawyer</i>	<i>1.0–5.0</i>	<i>Search sorting</i>
user_selection	<i>String</i>	<i>Selected section (Document Analysis / Chatbot / Lawyer Finder)</i>	<i>Enum value</i>	<i>Navigation control</i>
api_key_groq	<i>String</i>	<i>API key for Groq model</i>	<i>Hidden / Active / Expired</i>	<i>Model authentication</i>
api_key_hf	<i>String</i>	<i>API key for Hugging Face models</i>	<i>Hidden / Active / Expired</i>	<i>Model authentication</i>
session_id	<i>UUID</i>	<i>Unique ID for each user session</i>	<i>Active / Terminated</i>	<i>Session management</i>
error_message	<i>String</i>	<i>Stores latest system error message</i>	<i>None / Message Text</i>	<i>Exception handling and logs</i>
log_file	<i>File (.txt)</i>	<i>Log file maintaining error and event records</i>	<i>Open / Closed</i>	<i>System monitoring</i>
translation_lang	<i>String</i>	<i>Language for multilingual query translation (future scope)</i>	<i>English / Marathi / Hindi / Tamil</i>	<i>Translation module</i>
response_time	<i>Float (seconds)</i>	<i>Time taken to generate response</i>	<i>0.1–5.0s</i>	<i>Performance monitoring</i>
data_privacy_flag	<i>Boolean</i>	<i>Indicates whether data anonymization is applied</i>	<i>True / False</i>	<i>Security and compliance</i>
config.json	<i>File</i>	<i>Configuration file storing keys and settings</i>	<i>Exists / Missing</i>	<i>System initialization</i>

Appendix B - Group Log

B.1 Team Information

Name	Roll No.	Role / Responsibility
Soham Sunil Malvadkar	22141229	Legal Chatbot Model Integration, Coordination Report
Sanket Nivas Chopade	22141234	Team Leader, Backend Developer, Document Analysis Module, SRS Documentation
Abhijeet Vilas Ganjave	22141236	Database Developer, Lawyer Finder Module, UI Integration
Akash Ananda Khot	22141244	Data Collection, Testing, and FAISS Search Implementation
Sanket Nandkumar Adake	22143248	Front-End Developer, Streamlit Interface Design, Presentation & Diagrams

B.2 Meeting Minutes and Activities Log

Date	Mode	Duration	Agenda / Discussion Points	Decisions / Outcomes
25 Aug 2025	In-person	1.5 hrs	Brainstormed project ideas and identified problem statement on "Legal Assistance using AI."	Finalized project topic and team formation.
28 Aug 2025	Online (Google Meet)	1 hr	Researched existing systems (IBM Legal Chatbot, OpenAI Law models).	Defined three main modules: Document Analysis, Legal Chatbot, Lawyer Finder.
02 Sep 2025	In-person	2 hrs	Discussed AI models (Ollama, Groq, Hugging Face) and selection criteria.	Selected Ollama for document extraction and Groq API for generation.
05 Sep 2025	Online	1 hr	Planned data collection for marriage and employee law datasets.	Assigned roles for dataset gathering and preprocessing.
08 Sep 2025	In-person (Lab)	2 hrs	Implemented document extraction and	Successfully stored extracted data as .pkl files.

			vectorization using FAISS.	
12 Sep 2025	Online	1 hr	Configured Groq API and tested query-response flow.	Achieved working prototype for document Q&A.
15 Sep 2025	In-person	1.5 hrs	Integrated Hugging Face-based chatbot for legal domains.	Chatbot gave accurate legal responses with pretrained data.
18 Sept 2025	Online	1 hr	Developed Lawyer Finder database with filtering and rating features.	Integrated database with UI using Streamlit.
22 Sept 2025	In-person	2 hrs	Created front-end layout and combined all modules.	Full system integration achieved.
29 Sept 2025	Online	1 hr	Conducted end-to-end system testing.	Identified and fixed bugs in file upload and API handling.
3 Oct 2025	In-person	2 hrs	Drafted project synopsis and presentation slides.	Finalized presentation structure for seminar.
23 Sept 2025	Online	1 hr	Divided SRS sections among members.	Sanket C. to compile final SRS; others to review and edit.
27 Sept 2025	In-person	1 hr	Discussed feedback from guide and professor.	Incorporated changes in SRS and diagrams.
31 Oct 2025	Online	1 hr	Conducted final review meeting before submission.	Completed and finalized all documents and presentation.

B.3 Individual Contributions Summary

Team Member	Primary Contributions
Soham Malvadkar	Collected legal data, implemented chatbot logic, Groq API fine-tuning, drafted chatbot documentation.
Sanket Nivas Chopade	Developed document analysis pipeline, integrated FAISS retrieval, authored SRS sections, coordinated with professor.
Abhijeet Ganjave	Designed and populated lawyer database, developed search and filter logic, assisted in testing and debugging.
Akash Khot	Implemented vectorization and chunking logic, optimized model response speed, prepared performance testing results.
Sanket Adake	Designed Streamlit front-end, UI layout, and module switching navigation; compiled PowerPoint and visual diagrams.

B.4 Tools and Resources Used

- **Programming Language:** Python 3.8+
- **Frameworks:** Streamlit, FAISS, Hugging Face Transformers
- **APIs and Models:** Groq API (LLaMA 3.1-8B), Ollama DeepSeek R1 1.5B
- **Database:** SQLite / CSV
- **Development Tools:** PyCharm, Visual Studio Code, Google Colab
- **Version Control:** GitHub private repository
- **Communication Tools:** Google Meet, WhatsApp, Email

B.5 Summary of Group Collaboration

Throughout the development of the AI Legal Assistant, the group maintained a high level of collaboration through frequent online and in-person meetings. Each member contributed significantly to a specific subsystem while supporting integration and documentation efforts.

All deliverables (prototype, presentation, and SRS) were completed collaboratively and reviewed collectively before submission to ensure quality and completeness.